CIOS as of this morning: 47/95

Final Thurs, May 6 8:00-10:50 AM

Synchronization and Consistency

Synch needed for shared mem programs:

use same mechanisms

1. Mutex
   Only one thread can access a critical section
2. Barrier
   Synch all threads to known point

Primitives vs. Programming models

Mutex:

Global variable 'lock'

getlock(lock) block until 'lock' is acquired

unlock(lock) releases lock for other threads to acquire

Use

```
getlock(lock)
    critical section code
unlock(lock)
```

Implementing getlock

```
getlock(lock) {
    while (TS(lock) == 1)
        ;                    // spin
}
```

TS(lock) automically read lock and set lock to 1

```
unlock(lock) {
    lock = 0;
}
```

Atomic swap instruction

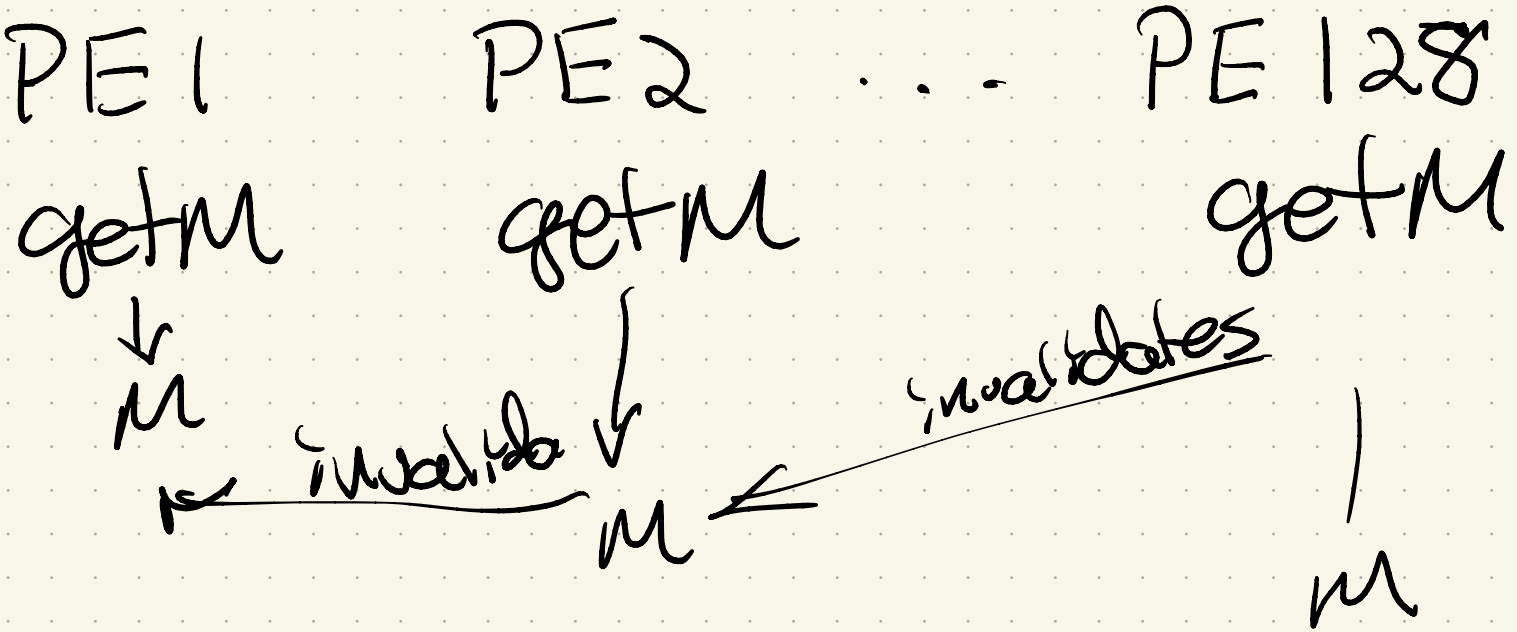
SWAP(lock, value)

exchanges lock for value

returns lock

```
TS(lock) {
    oldval = SWAP(lock, value);
    return oldval;
}
```

```
getlock(lock){
    while (TS(lock)==1)
        ;                    // spin
}
```



Test and Test and Set

```
TTS(lock) {
    oldval = lock; // load
    if (oldval == 0) {
        oldval = SWAP(lock, 1);
    }
    return oldval;
}

getlock(lock) {
    while (TTS(lock) == 1)
        ;                    // spin
}
```

Another primitive

Fetch and Op

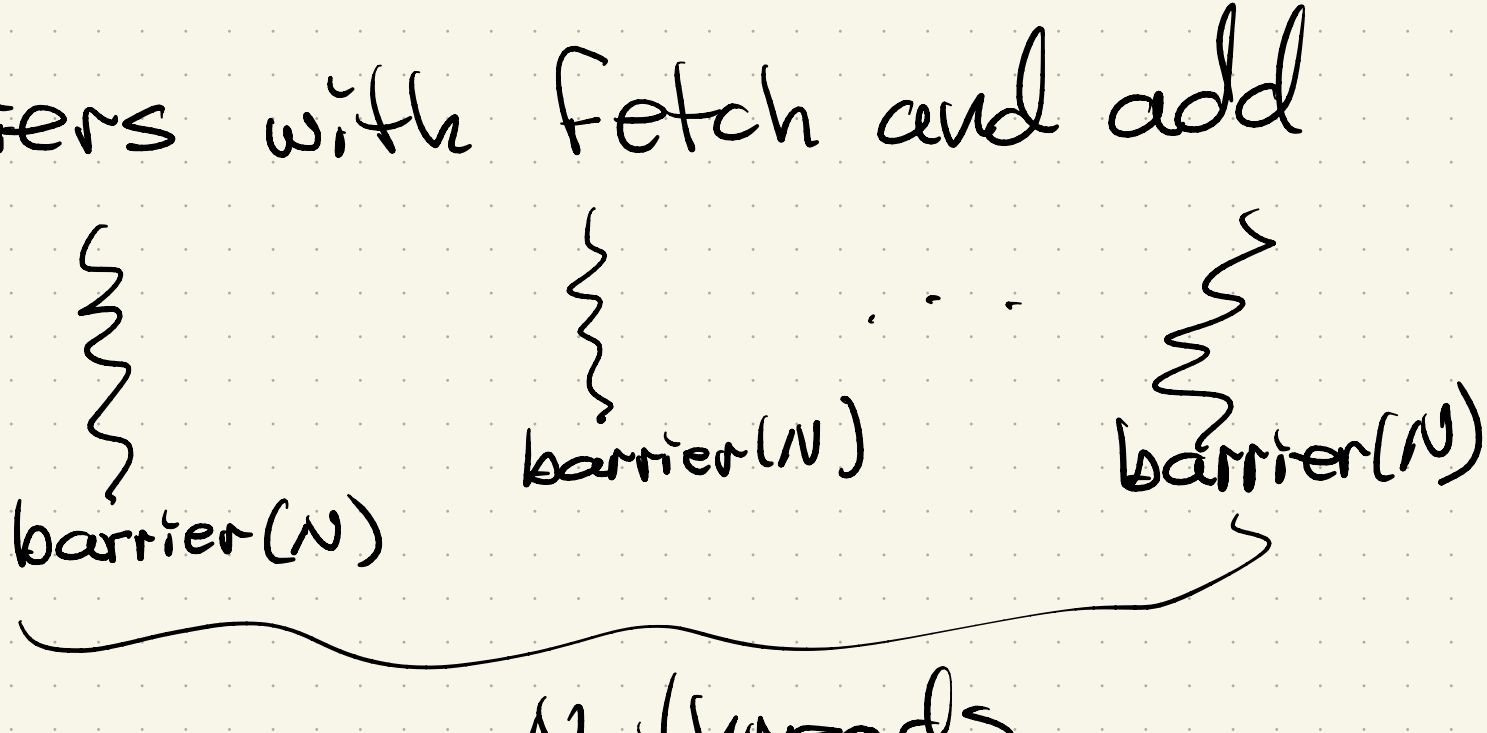
```
fetch and op (x, c) {
    x = x op c; //automically
    return (x);
}
```

Most commonly, op = '+'

```
fetch and add (x, c) {
    x = x + c;
    return (x);
}
```

could build from TTS, TS, SWAP

Barriers with fetch and add



counter=0

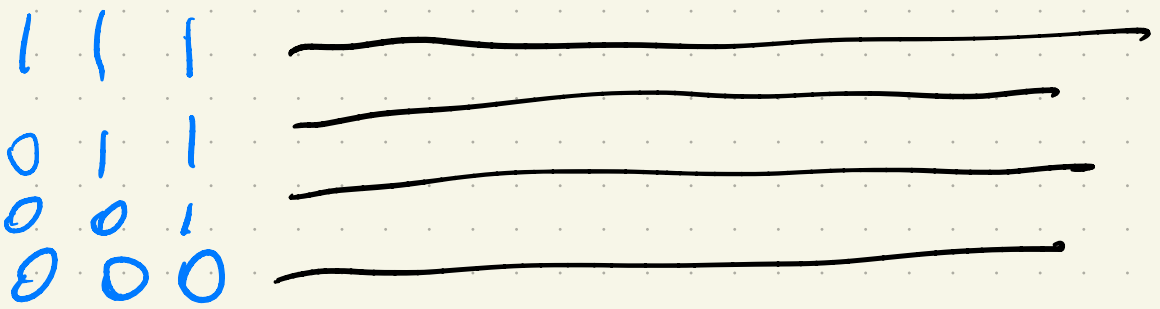
```
barrier(N) {
    if ( fetchandadd(counter, 1 ) < N) {
        while (counter < N)
            ;
    }
    counter = 0;
}
```

Barrier as a primitive

1-hot encoding for N threads,
(# threads = # PEs)

PE1	PE2	PE3	PE4
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

four shared wires on bus



when all lines = 1111, exit barrier

Aside: Parallel For and fetchandAdd

```
For (i=0; i<N; i++){
    A[i] = B[i] * C[i];
}

fork()
 while((iLocal = fetchandadd(i))<N)
    A[iLocal] = B[iLocal] * C[iLocal];
join() or barrier(...)
```

Another primitive - Load Linked & Store Conditional

LL R1, X :

R1 = Mem[X]

link = X // internal reg

⋮

if X is invalidated in $ then

link = 0

Store conditional

```
SC R1, X
    if (link == X) {
        Mem[X] = R1;
        R1 = 1 // success
    } else {
        R1 = 0
    }
```

```
getlock(lock)
    do {
        R2 = 1;
        LL R1, lock
        SC R2, lock
    } while (R2 == 0 || R1 != 0);
```

Alternatives to mutex synch

"Transaction"

PE i begins "transaction" XBEGIN

: x = y; :

ends "transaction" XEND

succeeded or failed

if failed, try again

can use coherence protocol to detect failure

if invalidate occurs, fail

Problem FF = DD = 0

PE1 PE2

D=1 FF = F

F=1 DD = D

~~DD = D~~

~~FF = F~~

(FF, DD) (0,0), (0,1), (1,1)

(1,0)

```
Flag = 0
X = 2
Y = 4
```

PE 1

```
X = Y
Flag = 1
```

PE 2

```
while (flag != 1)
    ;
Z = X;
```

Programmer expects Z = 4

But can get Z = 2

What if load of Y is a cache miss?

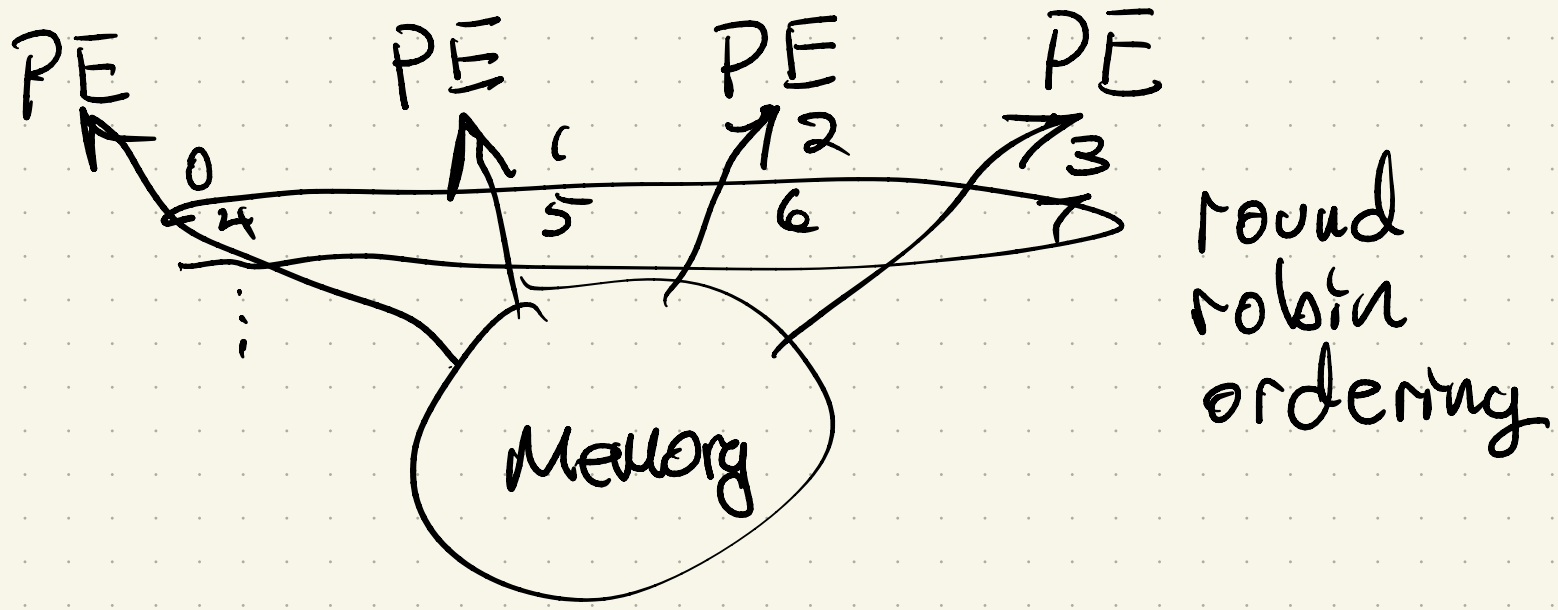
Memory consistency

Coherence is a guarantee about the same location in memory

Consistency is a guarantee about ordering of accesses between different locations

Strict consistency

1. All accesses executed by all PEs are kept in the same global order
   No read/write reordering
2. Accesses between PEs are ordered by a global clock
3. The same ordering is seen by all PEs



Sloow

Sequential Consistency

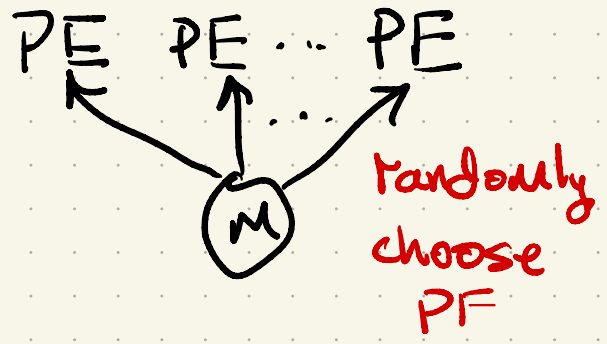
{ execute and detect inconsistency roll back

like strict consistency but accesses are seen in an order consistent with an interleaved sequential execution

X=Y=0
Z=0

X=1 ... Y=2 PE1

Z=Y+X PE2



all these outcomes are valid

Z=0, Z=1 or Z=3 } not Z=2